FDE ACADEMY · LESSON 3 OF 9

Prompt and Context Engineering

An LLM writes the recommendation, and every claim it makes can be checked.

Raunak Sharan
Enterprise Agentic AI Engineering

git checkout lesson-03

OBJECTIVES

By the end of today you can

01

Put every model provider behind one port, and run offline on a mock

02

Engineer the context: minimise, cite, prioritise

03

Ask for structured output and parse it robustly

04

Validate content: citations, numbers, consistency

05

Repair once, then fail safe to a human

06

Record model, prompt version and tokens with every proposal

THE PORT · LLM/CLIENT.PY

Nothing outside llm/ imports a vendor SDK

```
class LLMClient(Protocol):
model: str

def complete(
self, messages: list[Message], *,
max_output_tokens: int = 800,
json_mode: bool = False,
) → LLMResponse: ...

class LLMUnavailable(Exception):
"""timeout, outage, quota, no key"""
```

MockLLM (default)

Deterministic, offline, no key. Fault injection on demand.

LiteLLMClient

Any provider: LLM_MODE=live and LLM_MODEL=... Optional install.

CONTEXT ENGINEERING · AGENTS/CONTEXT.PY

Decide exactly what the model sees

Minimise

No name, date of birth, ID digits, email or address. What the model never sees, it can never leak or be biased by.

Give everything an id

```
metric:post_loan_dti
signal:IDENTITY_MISMATCH
doc:…/payslip.txt
mdl-upl-policy-3.2#8
```

Prioritise

Signals, then metrics, then policy excerpts. Over budget? Policy goes first, lowest priority last.

```
uv run underwriter prompt data/applications/APP-006.json → 3,809 chars · 17 citable ids
```

THE PROMPT · PROMPTS/UNDERWRITING_V1.MD

Five rules and one worked example

1. Use only the facts in <case_json>. Never invent a number, score or policy.

2. Back every claim with an id from the context.

3. Copy every number you rely on into “figures”, exactly.

4. Review signals or missing information mean MANUAL_REVIEW.

5. Reply with one JSON object, these keys, nothing else.

The prompt says the model **proposes** and the engine decides. That’s a courtesy: the architecture doesn’t depend on the model obeying it.

STRUCTURED OUTPUT · AGENTS/VALIDATION.PY

A strict schema, parsed forgivingly

```
class UnderwritingOutput(BaseModel):
model_config = ConfigDict(extra="forbid")

recommendation: Literal["APPROVE",
"DECLINE", "MANUAL_REVIEW"]
rationale: str # ≤ 800 chars
risk_factors: list[str]
missing_information: list[str]
evidence: list[str] # ≥ 1 id
figures: dict[str, float]
```

Strict

No extra keys, no unknown outcomes, at least one citation.

Forgiving parse

Strip code fences and “Sure! Here is…” chatter before parsing.

SEMANTIC VALIDATION

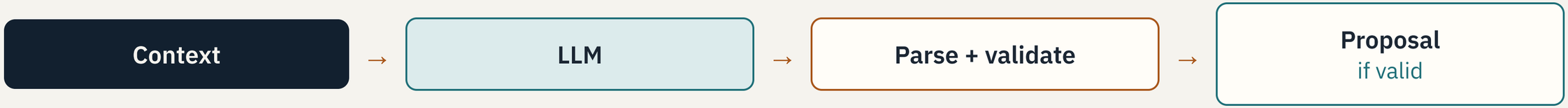
Parsing is not the same as correct

Check	Example error message (sent back to the model)
Citations exist	evidence cites ids not in the context: ['policy:does-not-exist']
Figures match facts	figures.post_loan_dti = 0.3559 but the fact is 0.4559
Prose numbers match	rationale states 35.6% which matches no fact in the context
Missing info → review	information is missing but the recommendation is not MANUAL_REVIEW
Signals → no approve	APPROVE despite open review signals: ['IDENTITY_MISMATCH']

Each message is specific enough to fix. That’s what makes one repair attempt worth sending.

THE LOOP · AGENTS/UNDERWRITING.PY

Repair once, with the exact errors



Invalid once

Send back the model’s own answer plus the error list. Ask for corrected JSON only.

Invalid twice

Stop. MANUAL_REVIEW with INVALID_MODEL_OUTPUT. More retries just spend tokens.

Provider down

No retry here: MODEL_UNAVAILABLE. Retries with backoff belong to lesson 8.

FAILING SAFE · LLM/MOCK.PY FAULTS

Every failure lands somewhere defined

Fault (--llm-fault)	Attempts	APP-001 (clean)	APP-005 (policy decline)
none	1	Approve	Decline
malformed · bad_citation · wrong_number	2	Approve (repaired)	Decline
*_always	2	Manual review · INVALID_MODEL_OUTPUT	Decline · policy still applies
unavailable	1	Manual review · MODEL_UNAVAILABLE	Decline · policy still applies

A clear policy decline never depended on the model, so a model outage can’t change it. Everything else goes to a human with a reason code.

LIVE WALKTHROUGH

One case, four outcomes

```
uv run underwriter check-llm
uv run underwriter prompt data/applications/APP-006.json
uv run underwriter run data/applications/APP-002.json
uv run underwriter run …/APP-002.json --llm-fault malformed
uv run underwriter run …/APP-002.json --llm-fault wrong_number_always
uv run underwriter run …/APP-001.json --llm-fault unavailable
```

```
Model: mock-underwriter-v1 · prompt underwriting-v1 · attempts 2 · INVALID:
figures.post_loan_dti = 0.3559 but the fact is 0.4559
```

LAB · 100 MINUTES

Three exercises

35 MIN · CODE

A new validator

A DECLINE must cite at least one policy chunk. Write the failing test first, then the check.

30 MIN · CONTEXT

Budget and priority

Render APP-006 at 6000, 3000 and 2000 chars. Is the drop order right? Change it and defend the change in two sentences.

35 MIN · PROMPT

Prompt v2

Make one deliberate change, bump PROMPT_VERSION, run the suite. With a key, compare v1 and v2 live.

HOMEWORK AND INTERVIEW PREP

What strong work looks like

Criterion	Strong	Weak
Validators	Check content against facts	Check fields exist
Repair	One retry, specific errors	Retry until it works
Failure	Typed reason, human review	Exception or default
Context	Minimal, cited, budgeted	Whole application
Provenance	Model, prompt, tokens kept	Unknown prompt

Interview questions

- Validate LLM output beyond JSON schema?
- Keep PII out of prompts, and why it matters
- Provider down or over quota: what now?
- Version prompts; trace a decision to one
- Retry policy for malformed output

TAKEAWAY

Trust the model’s output only as far as you can check it.

Next · Lesson 4

Tool-using agents

The agent calls bureau, bank and policy tools itself, inside a step cap, against tools that fail on purpose.